

Prompt-Based Text-to-SQL Semantic Parsing with Schema Serialization Strategies

Ayşenur Savaş¹ and Mustafa [SOYADI]¹

Department of Computer Engineering, Izmir Institute of Technology (IYTE), Izmir,
Turkey

{aysenursavas, mustafa.[soyadi]}@iyte.edu.tr

1 Project Title, Group Members, and Problem Definition

Project Title: Prompt-Based Text-to-SQL Semantic Parsing with Schema Serialization Strategies

Group Members:

- Ayşenur Savaş
- Mustafa [SOYADI]

Project Number: 12 — Text-to-SQL Semantic Parsing

1.1 Problem Definition

Text-to-SQL semantic parsing is the task of automatically translating natural language questions into executable SQL queries over relational databases. This task is central to building natural language interfaces to databases (NLIDB), enabling non-expert users to query structured data without knowledge of SQL syntax [?].

Task formulation:

- **Input:** A natural language question q and a database schema S
- **Output:** An executable SQL query y that answers the question
- **Task type:** Conditional generation (sequence-to-sequence)

The task is technically challenging due to several factors: (1) the compositional and hierarchical nature of SQL (nested queries, joins, aggregations), (2) the need for accurate *schema linking* between question entities and database columns/tables, and (3) the requirement for *cross-domain generalization* where models must handle unseen database schemas at test time [?].

We use the **Spider benchmark** [?] as our primary dataset and Google Gemini [?] as the language model for prompting-based baselines. In the final submission, we plan to incorporate instruction-tuned fine-tuning and compare it against prompting strategies.

Our project title has not changed since the proposal stage.

2 Dataset / Benchmark Status

We use the **Spider** benchmark [?], a large-scale, cross-domain Text-to-SQL dataset.

Table 1. Spider dataset statistics.

Property	Value
Dataset name	Spider (xlangai/spider on HuggingFace)
Data source	HuggingFace Datasets hub
Training examples	7,000
Validation examples	1,034
Number of databases	200+ across 138 domains
Input format	Natural language question + database schema
Output format	SQL query string

Preprocessing steps completed:

- Dataset downloaded and loaded via the HuggingFace **datasets** library
- Database schema information extracted by parsing SQL queries across the training set (table and column names identified per database)
- Three schema serialization formats implemented (plain text, CREATE TABLE syntax, compact notation)
- Zero-shot and few-shot prompt templates constructed

Known challenges:

- The HuggingFace version of Spider does not include explicit schema meta-data (**tables.json**); we reconstruct schema information by parsing SQL queries from the training set
- Schema linking remains imperfect for complex multi-table queries
- Some SQL queries involve nested subqueries and complex aggregations that are difficult to generate via prompting alone

The dataset has been **downloaded, cleaned, and prepared** for experiments.

3 Literature Review Progress

We have reviewed the following key areas relevant to our project:

3.1 Sequence-to-Sequence Approaches for Text-to-SQL

Early neural approaches to Text-to-SQL adopted sequence-to-sequence architectures. Zhong et al. [?] introduced Seq2SQL, which used reinforcement learning

to generate SQL from natural language over single tables. The Spider benchmark [?] later introduced cross-domain evaluation, requiring models to generalize to unseen databases. These works established the standard task formulation and evaluation metrics (Exact Match and Execution Accuracy) that we use in our project.

3.2 Pre-trained Language Models for Text-to-SQL

The introduction of large pre-trained language models has substantially advanced Text-to-SQL performance. BRIDGE [?] proposed a hybrid approach combining pre-trained representations with schema linking. UnifiedSKG [?] demonstrated that multi-task learning across structured knowledge grounding tasks improves Text-to-SQL accuracy. More recently, large language models (LLMs) such as GPT-4 and Gemini have shown strong zero-shot and few-shot performance on Text-to-SQL benchmarks [?,?]. DAIL-SQL [?] achieved competitive results with carefully designed prompting strategies, demonstrating that prompt engineering is a viable alternative to fine-tuning for this task.

3.3 Schema Encoding and Linking

Effective schema encoding is critical for cross-domain generalization. RAT-SQL [?] introduced relation-aware attention for encoding schema structures. PICARD [?] augmented T5 with constrained decoding to ensure syntactic validity. Our project focuses on a simpler approach: comparing different schema *serialization* formats within prompts, which to our knowledge has not been systematically studied with instruction-tuned LLMs.

Positioning of This Work. Unlike prior approaches that rely on complex graph-based schema linking, we adopt a prompting-based framework with Gemini and systematically compare schema serialization strategies. We plan to extend this with instruction tuning for the final submission.

4 Baseline Models or Prompting Strategies

We have designed and implemented two baseline prompting strategies:

4.1 Baseline 1: Zero-Shot Prompting

We evaluate Google Gemini (1.5 Flash) [?] in a zero-shot setting. The prompt provides the database schema and the natural language question without any SQL examples. This establishes the floor performance achievable by prompting alone without task-specific demonstrations.

Listing 1.1. Zero-shot prompt template

```
You are an expert SQL developer. Your task is to
translate the natural language question into SQL.
```

```

Database: {db_id}
Schema: {serialized_schema}
Question: {question}

Return ONLY the SQL query, nothing else.

```

4.2 Baseline 2: Few-Shot Prompting (3-shot)

We extend the zero-shot baseline by providing three SQL examples before the target question. Examples are selected from the same database when available, otherwise generic examples are used. This tests whether in-context learning with examples improves SQL generation quality.

Current status:

- Both baselines are **fully implemented** in our evaluation pipeline (`evaluate.py`)
- Schema serialization supports three formats (Format A: plain text, Format B: CREATE TABLE, Format C: compact notation)
- API quota limitations are currently being resolved; initial test runs have been conducted
- Systematic evaluation on the full validation set is in progress

5 Initial Experimental Results

Due to API quota limitations encountered during the evaluation period, we report results from our initial test runs on a small subset. Full evaluation on the Spider validation set is in progress and will be completed for the final submission.

Table 2. Preliminary results on Spider validation subset. EM = Exact Match (%).

Method	Schema	EM (%)	Status
Zero-shot (Gemini 1.5 Flash)	Format A	–	In progress
Zero-shot (Gemini 1.5 Flash)	Format B	–	In progress
Few-shot 3-shot (Gemini 1.5 Flash)	Format A	–	In progress
Few-shot 3-shot (Gemini 1.5 Flash)	Format B	–	In progress
Instruction-tuned (planned)	Best	–	Not started

Preliminary observations: From our initial API tests, we observed that:

- Zero-shot prompting produces syntactically valid SQL for simple queries but struggles with multi-table joins and nested subqueries
- Schema information significantly affects output quality; omitting the schema leads to frequent hallucination of non-existent table names

- Few-shot examples help the model follow the expected output format more consistently

We expect to obtain complete numerical results once API access is fully configured.

6 Planned Improvements and Technical Direction

For the next phase, we plan the following improvements:

1. **Complete baseline evaluation:** Run both baselines on the full Spider validation set (1,034 examples) with all three schema formats
2. **Instruction-tuned fine-tuning:** Fine-tune a smaller model (Flan-T5-base [?] or similar) with LoRA [?] on Spider training data, using the best-performing schema format
3. **Improved schema extraction:** Download and integrate Spider’s original `tables.json` for accurate schema information including primary and foreign keys
4. **Better few-shot selection:** Implement similarity-based example selection (selecting examples whose questions are semantically similar to the target)
5. **Post-processing:** Add SQL syntax validation and basic error correction as a post-processing step
6. **Execution accuracy:** Implement execution-based evaluation by running generated SQL against actual databases

7 Ablation or Prompt Sensitivity Plan

Our ablation study will systematically compare the following dimensions:

Table 3. Planned ablation study dimensions.

Dimension	Variants
Schema format	Format A (plain text) vs. B (CREATE TABLE) vs. C (compact)
Prompting strategy	Zero-shot vs. 1-shot vs. 3-shot vs. 5-shot
Temperature	0.0 vs. 0.3 vs. 0.7
Model size	Gemini 1.5 Flash vs. Gemini 1.5 Pro (if available)
With/without fine-tuning	Prompting only vs. LoRA fine-tuned

The primary research question is: *How does schema serialization format affect Text-to-SQL accuracy across different prompting strategies?* We hypothesize that Format B (CREATE TABLE syntax) will perform best with code-oriented models, as it resembles the input distribution seen during pre-training.

8 Error Analysis and Generation Quality Analysis Plan

For the final report, we plan to conduct a detailed error analysis by:

1. **Manual inspection** of at least 50 failure cases from the validation set
2. **Categorization** of errors into the following types:
 - Schema linking errors (wrong table or column selected)
 - Incorrect JOIN conditions
 - Aggregation errors (COUNT, SUM, AVG misuse)
 - Nested query failures (missing or incorrect subqueries)
 - WHERE clause mismatches (wrong conditions or values)
 - Hallucinated table/column names
3. **Difficulty-level analysis:** Comparing performance across Spider’s difficulty categories (easy, medium, hard, extra hard)
4. **Generation quality:** Evaluating syntactic validity, semantic correctness, and query efficiency

We will also examine cases where the generated SQL produces correct results but differs from the gold SQL (valid alternatives), which is captured by execution accuracy but missed by exact match.

9 Ethical Considerations and Bias

Deploying Text-to-SQL systems introduces several ethical considerations:

Hallucination Risk: LLMs may generate syntactically valid SQL that queries non-existent columns or produces incorrect results silently. Non-expert users cannot easily detect such errors, which could lead to incorrect data-driven decisions.

SQL Injection Risk: Integrating user-provided free-text into SQL generation pipelines could introduce injection vulnerabilities if outputs are executed without sanitization.

Domain and Language Bias: Spider’s training data is English-only and skews toward standard business domains (universities, music, sports). Models trained on Spider may perform poorly on multilingual queries or specialized domains such as healthcare or legal databases, raising fairness concerns.

API Usage: We use the Google Gemini API for inference. All API calls use standard, publicly available endpoints, and no private or sensitive data is sent through the API. We are mindful of responsible API usage and rate limiting.

10 GitHub and Reproducibility Status

Repository: <https://github.com/Aysenursvs/CENG467-Text-to-SQL>

Current repository structure:

Listing 1.2. Project directory structure

```
CENG467 - Text - to - SQL /
```

```

src/
  data_prep.py      # Dataset download and exploration
  evaluate.py       # Baseline evaluation pipeline
  utils.py          # Schema extraction, prompts, metrics
  train.py          # (Planned) Fine-tuning script
data/               # Dataset statistics
results/            # Evaluation results (JSON)
notebooks/          # Exploration notebooks
report/             # This LaTeX report
requirements.txt    # Python dependencies
README.md           # Setup and usage instructions
.env                # API configuration (not committed)
.gitignore

```

Reproducibility checklist:

GitHub repository created and actively maintained
 README with setup instructions and usage commands
 requirements.txt with all Python dependencies
 .env template for API key configuration (documented in README)
 .gitignore excluding data, results, and sensitive files
 Consistent commit history (8+ commits as of checkpoint)
 Automated evaluation script with command-line arguments
 Full results reproduction with single command

11 Current Challenges and Next Steps

11.1 Current Challenges

- **API quota limitations:** The free-tier quota of the Gemini API has been exhausted during development, delaying systematic evaluation. We are exploring alternative API plans and model endpoints.
- **Schema extraction:** The HuggingFace Spider dataset does not include explicit schema metadata; we reconstruct schema information by parsing SQL queries, which is imperfect for databases with tables not referenced in the training queries.
- **Computational constraints:** Fine-tuning is planned using a personal computer without a dedicated GPU, which may require using parameter-efficient methods (LoRA/QLoRA) or cloud-based resources (Google Colab).
- **Evaluation complexity:** Exact Match is a strict metric that may underestimate actual performance; implementing Execution Accuracy requires setting up individual SQLite databases for each Spider example.

11.2 Next Steps

1. Resolve API access and complete full baseline evaluation (Week 12)

2. Download and integrate Spider’s original schema files for improved schema encoding (Week 12)
3. Implement and evaluate instruction-tuned fine-tuning with LoRA (Week 13)
4. Conduct ablation study across schema formats and prompting strategies (Week 13)
5. Perform error analysis on at least 50 failure cases (Week 14)
6. Write final report and prepare project demonstration (Week 14–15)

References